TalentIQ

TECHNICAL DESIGN DOCUMENT

AI-assisted resume screening and client shortlist portal

GitHub: https://github.com/Sankalp-gupta1/TalentIQ-AI-Resume-Screening

Live application: Not deployed — local execution only

Status: Full-stack take-home / portfolio MVP

Primary users: Staffing recruiters and client hiring managers

Document scope

This document describes the implementation in the repository and separates implemented MVP behavior from future production work. No hosted deployment or API secret is included.

Human-in-the-loop principle

A candidate match score is advisory only. It must not automatically reject a candidate or replace a recruiter's review of the original CV.

GitHub repository: https://github.com/Sankalp-gupta1/TalentIQ-AI-Resume-Screening **Live application URL:** Not deployed — local execution only **Product:** AI-assisted resume screening and client shortlist portal **Status:** Full-stack take-home / portfolio MVP **Primary users:** Staffing recruiters and client hiring managers

1. Executive Summary

TalentIQ addresses a staffing workflow in which recruiters manually compare resumes with job descriptions, maintain shortlists in spreadsheets, and coordinate client feedback across disconnected channels. The MVP provides a central workflow for job intake, editable scorecards, resume parsing, evidence-based candidate ranking, client review and pipeline visibility.

The system is designed as a human-in-the-loop assistant. A numeric score is advisory, not an automated hiring decision. Recruiters remain responsible for inspecting the original CV and deciding whether to submit a candidate.

2. Problem and Product Goals

Current problem

- Resume screening consumes recruiter time.
- Candidate information is spread across files and spreadsheets.
- Client feedback and interview status are difficult to track consistently.
- A score without supporting evidence is difficult to trust.

Product goals

- Convert a job description into a structured, editable scorecard.
- Accept PDF and DOCX resumes and extract usable text.
- Detect exact-file duplicates within a requisition.
- Rank profiles using explainable criteria and evidence excerpts.
- Let clients review only candidates shared with their organization.
- Track shortlist, rejection reason, proposed interview slot and pipeline events.
- Provide basic consent, deletion and audit foundations.

3. Users, Roles and Permissions

Role	Capabilities	Restrictions
Recruiter	Create jobs, edit scorecards, upload CVs, inspect ranked candidates/evidence, submit profiles, view pipeline/audit, process deletion	Must review original CV before client submission
Client hiring manager	Review profiles submitted to their tenant, shortlist, reject with reason, propose interview slot	Cannot browse unsubmitted profiles; contact details and raw CV text are masked

Client access is associated with `client_id` on the user and submission. The API filters client submissions by the authenticated user's tenant.

4. Scope and Screen Design

- **Authentication:** JWT login with role-aware application data.
- **Dashboard:** summary counts and open requisitions.
- **Job scorecard:** job description intake and editable must-have skills, experience range, location, notice limit and CTC.
- **Resume intake:** multi-file PDF/DOCX upload, validation, duplicate detection and per-file status.
- **Candidate discovery:** ranked profiles, search and evidence details.
- **Client portal:** submitted candidates only, masked contact information and client decision actions.
- **Pipeline:** chronological events for screened, submitted and client decision stages.
- **Audit & privacy:** access-event review and recruiter deletion workflow.

5. Architecture and Technology Choices

```
Browser
  | Next.js 15 / React 19 / TypeScript
  | REST + multipart
  v
FastAPI / Pydantic / JWT
  | Optional OpenAI JD extraction
  | PDF/DOCX parsing + profile extraction
  | Evidence-based rules scoring
  | Role / tenant authorization
  v
SQLAlchemy 2 --> PostgreSQL 16 (Docker) / SQLite (direct local backend)
  | Users, Jobs, Candidates, Scores, Submissions
  | PipelineEvents, AuditLogs

Redis + Celery worker scaffold --> future asynchronous ingestion
```

Technology decisions

- **Next.js + TypeScript:** component-based UI with a typed client.
- **FastAPI:** REST API, request validation and interactive OpenAPI docs.
- **SQLAlchemy:** relational persistence and explicit entity relationships.
- **PostgreSQL:** primary Docker database; SQLite supports a quick backend-only local run.
- **PyMuPDF / python-docx:** selectable text extraction from PDF and DOCX.
- **Rules scorer:** deterministic, low-cost and explainable baseline.
- **OpenAI API:** optional structured extraction of job requirements only.
- **Redis/Celery:** background-job foundation; not a completed bulk-processing pipeline.

6. Data Model

Entity	Important fields	Purpose
User	name, email, password_hash, role, client_id	Recruiter/client identity
Job	title, location, description, experience range, required_skills, notice limit, CTC	Requisition and scorecard
Candidate	candidate_code, parsed profile, contact fields, resume hash/path, consent/deleted flags, job_id	Candidate profile and source CV reference
Score	candidate_id, job_id, score, engine, evidence, explanation	Advisory score and criterion evidence
Submission	candidate_id, job_id, client_id, status, reason, interview_slot	Client-facing submission and decision
PipelineEvent	candidate_id, job_id, status, note, created_at	Workflow history
AuditLog	user_id, action, resource_type, resource_id, created_at	Sensitive profile access history

Models are implemented in `backend/app/models.py`; Alembic migration files are included.

7. Key Data Flows

7.1 Job description intake

- Recruiter submits a description and initial fields.
- If `OPENAI_API_KEY` is configured, the backend requests structured job fields from the configured model.
- If the key is absent or the provider call fails, manual/default values remain.
- Recruiter reviews and edits the scorecard.

7.2 Resume upload and parsing

- Recruiter selects PDF/DOCX files and confirms the hiring-purpose consent condition.
- API checks extension, configured batch count and 12 MB per-file size limit.
- API computes SHA-256 and checks for an exact duplicate within the same job.

- PyMuPDF or python-docx extracts selectable text.
- Parser extracts common profile fields: name, role, skills, experience, notice and contact details.
- Empty/scanned content is returned as `needs_ocr`; it is not silently scored.
- Candidate and score records are stored and a `screened` pipeline event is created.

7.3 Candidate ranking

The rules scorer compares parsed candidate attributes against the job scorecard. It produces a 0–100 advisory score and criterion-level evidence. It does not automatically reject or hide a candidate.

7.4 Client review

- Recruiter submits a candidate to a client tenant.
- Client API returns only submissions associated with the authenticated client's `client_id`.
- Email, phone and raw resume text are masked from client profile detail.
- Client can shortlist, reject with a required reason, or propose an interview slot.
- The decision is recorded as a pipeline event.

7.5 Deletion and audit

Candidate reads are logged. The recruiter deletion endpoint anonymizes selected profile fields and removes client submissions. Production must also remove uploaded files and derived data from every storage/backup location according to a retention policy.

8. AI and Scoring Design

8.1 Optional AI extraction

OpenAI is optional and used by `backend/app/ai.py` for structured job-description extraction only. The key is supplied through the operator's private `.env` and is not hardcoded or exposed to the browser.

8.2 Candidate scoring implementation

Current implementation: deterministic Python rules scorer, labelled `rules_fallback`. It is not an LLM-generated CV score or trained ML model.

Criterion	Maximum points	Current approach
Required skills	40	Points distributed across must-have skills; checks extracted skills and resume text
Experience	30	Full points in configured range; partial/unknown treatment otherwise
Notice period	20	Full points within limit; partial/unknown if over or missing
Location	10	Exact text match; otherwise partial/unknown credit

Each criterion stores a status and short evidence excerpt. Evidence must be checked against the original CV because parsing can be imperfect.

8.3 Human oversight

- No automatic rejection threshold is implemented.
- Low-scoring profiles remain in the recruiter list/search.
- Recruiters must inspect the original CV before sharing.
- Scores are decision-support signals, not predictions of job performance.
- A future iteration should compare weekly ranking samples with recruiter decisions and review false negatives and criterion-level disparities.

9. Privacy, Fairness and Security

Current MVP safeguards

- Scoring does not use candidate name, email or phone.

- Scorer evaluates skills, experience, notice and location only.
- Client users are limited to submissions for their own `client_id`.
- Client profile detail masks email/phone and omits raw CV text.
- Candidate profile access generates audit events.
- Upload endpoint requires recruiter confirmation of candidate consent for the hiring purpose.
- Recruiter deletion endpoint anonymizes selected fields and removes submissions.
- `.env` and local uploads are excluded from Git.

Important limitations

The consent checkbox is a recruiter attestation, not a complete candidate-facing consent/withdrawal system. The deletion endpoint does not establish erasure from every backup or external storage system. This project is not legal advice or a DPDP compliance certification.

Before real candidate data is used, add verifiable consent and notice, purpose limitation, retention/deletion schedules, encrypted private file storage, malware scanning, access reviews, deletion propagation, backups and legal/privacy review.

10. Bulk Processing and Performance Trade-offs

The API configuration allows the upload count to be set (default 500), and each file is size-checked. However, the current endpoint processes files synchronously in the request. Celery/Redis is a scaffold, not a complete per-file queue with progress, retries and dead-letter handling. The project must not be represented as production-ready or load-tested for 500-resume batches. Production should stream to private object storage and process each file asynchronously.

11. Assumptions and Out-of-Scope Items

Assumptions

- One staffing organization operates the recruiter workspace.
- A client user belongs to one client tenant.
- Recruiters explicitly select candidates to share.
- Sample profiles are fictional.
- Notice period is expressed in days.
- Scorecards are reviewed and editable before use.

Out of scope

- Job-board scraping and job-board integrations
- ATS/HRIS integrations
- Video interviews and background verification
- Calendar booking and email delivery
- Production OCR service
- Production-scale asynchronous bulk processing

12. Testing and Verification

Backend unit tests are under `backend/tests/` for resume parsing and scoring. A GitHub Actions workflow is included. Run:

```
cd backend
pip install -r requirements.txt
PYTHONPATH=. pytest -q
python -m compileall -q app
```

Frontend build check:

```
cd frontend
npm install
npm run build
```

Unit tests do not equal complete browser testing, performance testing, penetration testing or production readiness. No live deployment has been performed.

13. Local Setup and Configuration

Recommended Docker setup:

```
# Create private local configuration
cp .env.example .env
# Build and start
docker compose up --build -d
# Seed fictional demo records
docker compose exec backend python -m app.seed
```

On Windows PowerShell, use `Copy-Item .env.example .env` for the first command. Open frontend at `http://localhost:3000`, API docs at `http://localhost:8000/docs`, and health at `http://localhost:8000/api/v1/health`.

The optional `OPENAI_API_KEY` remains blank by default. See `deployment.md` for OS-specific instructions, demo credentials and environment variables.

14. Trade-offs and Next Steps

Current trade-off	Recommended next step
Synchronous ingestion	Queue, progress state, retry and dead-letter handling
Selectable text only	OCR with confidence thresholds and recruiter review
Heuristic parsing	Section-aware parsing and editable parsed fields
Rules-based scoring	Evaluated semantic matching with traceable evidence and recruiter labels
Basic consent attestation	Candidate-facing consent, withdrawal and purpose records
Local file volume	Encrypted private object storage and malware scanning
Basic audit log	Append-only audit retention, export and monitoring
MVP tenant checks	Comprehensive authorization and tenant-isolation tests

15. Repository and Delivery

- **Source repository:** https://github.com/Sankalp-gupta1/TalentIQ-AI-Resume-Screening
- **Live URL:** Not deployed by design; run locally.
- **API key:** No key is committed. Each operator may add their own key to their private `.env`.
- **Submission documents:** `TDD.pdf` and `deployment.md`.

This document describes the implementation in the repository and distinguishes implemented behavior from planned production work.